1. Abstract

This project presents the development of an **end-to-end Retrieval-Augmented Generation (RAG) chatbot** designed to answer questions based on Sanskrit documents. The system integrates **hybrid information retrieval techniques (semantic search and keyword search)** with a **Large Language Model (LLM)** to generate context-grounded responses.

The chatbot supports both **Devanagari Sanskrit input and Latin transliteration**, making the system accessible to users unfamiliar with the Devanagari script. The architecture combines document retrieval with generative AI to ensure that responses are derived from verified document sources rather than relying solely on model knowledge.

The system is implemented using **FastAPI for backend services**, **FAISS for vector similarity search**, **BM25 for keyword retrieval**, and the **Qwen Large Language Model for answer generation**.

2. Introduction

Large Language Models (LLMs) have significantly improved natural language understanding and generation. However, these models often generate **hallucinated or ungrounded responses** because they rely only on their training data and do not access external knowledge during inference.

Retrieval-Augmented Generation (RAG) systems address this limitation by combining **document retrieval with generative models**. Instead of generating answers solely from internal knowledge, the model retrieves relevant information from a document corpus and uses that information to produce grounded responses.

This project implements a **Sanskrit Question Answering system** using a hybrid retrieval approach and an LLM-based generation mechanism. The system is deployed with a web interface and API backend, enabling users to interactively query Sanskrit documents.

3. Problem Statement

Traditional language models face several limitations when applied to document-based question answering:

• They do not have direct access to external knowledge sources during inference
• They may generate incorrect or hallucinated answers
• They cannot easily perform question answering over structured Sanskrit documents

To address these limitations, a **Retrieval-Augmented Generation chatbot** was designed.

Objective

The goal of this project is to build a system that:

• Accepts queries in **Sanskrit or transliterated text**
• Retrieves relevant passages from Sanskrit documents
• Generates context-grounded answers using retrieved information
• Minimizes hallucinated responses through retrieval grounding

4. System Architecture Overview

The system consists of two main pipelines:

1. Offline Pipeline (Knowledge Base Creation)

This pipeline prepares the document knowledge base before deployment.

2. Online Pipeline (Question Answering)

This pipeline processes user queries in real time.

The architecture follows the standard **RAG workflow**, combining document indexing, hybrid retrieval, and LLM-based answer generation.

5. Offline Pipeline – Knowledge Base Creation

The offline pipeline is responsible for preparing the searchable document index.

Step 1: Document Loading

The system supports multiple document formats including:

• TXT
• DOCX
• PDF

All documents are converted into plain text to enable processing.

Step 2: Text Chunking

Large documents are divided into smaller segments to improve retrieval accuracy.

Configuration used:

Chunk Size: **800 characters**
Chunk Overlap: **150 characters**

Chunk overlap preserves semantic continuity between adjacent text segments.

Step 3: Embedding Generation

Each text chunk is converted into a **vector embedding** using a multilingual embedding model.

Embedding Model:

intfloat/multilingual-e5-base

These embeddings represent semantic meaning and enable similarity search.

Step 4: Vector Index (FAISS)

All chunk embeddings are stored in a **FAISS vector database**, which allows fast similarity search between user queries and document chunks.

Purpose:

• Semantic similarity retrieval
• Fast nearest-neighbor search

Step 5: Keyword Index (BM25)

In addition to vector search, chunks are indexed using **BM25 keyword-based retrieval**.

BM25 retrieves documents based on exact word matches and complements vector similarity search.

Offline Output

Component	Purpose
FAISS Index	Semantic search
BM25 Index	Keyword search

Together they enable **hybrid retrieval**.

6. Online Pipeline – Question Answering

This pipeline executes when a user submits a query.

Step 1: User Input

The user enters a query through the web interface.

Supported formats:

• Sanskrit (Devanagari script)
• Latin transliteration

Example query:

ghantakarNasya katha kathaya

Step 2: Transliteration Detection

If the query is written in Latin characters:

• The system detects the script
• Converts it to Devanagari before retrieval

This ensures consistent search in the document corpus.

Step 3: Hybrid Retrieval

Two retrieval methods operate simultaneously:

Retriever	Function
FAISS	Semantic similarity search
BM25	Keyword matching

The results are combined using a weighted scoring approach:

Hybrid Score = **0.6 × FAISS + 0.4 × BM25**

This improves:

• Recall (finding relevant passages)
• Precision (removing irrelevant results)

Step 4: Context Construction

The top retrieved chunks are merged to form a **context block**.

This context is passed to the language model along with the user query.

Step 5: LLM-Based Answer Generation

The system uses the **Qwen Large Language Model** to generate answers.

Prompt rules:

• Use only the provided context
• Do not add external knowledge
• Generate answers in Sanskrit
• If no answer is found, return an appropriate response

This ensures answers remain **grounded in retrieved documents**.

Step 6: Output Formatting

If the user submitted the query using Latin transliteration:

• The generated Sanskrit response is converted back to Latin script.

The final answer is returned to the web interface.

7. Technology Stack

Layer	Technology
Backend API	FastAPI
Frontend	HTML, CSS
LLM	Qwen
Embedding Model	multilingual-e5-base
Vector Database	FAISS
Keyword Retrieval	BM25
Text Chunking	RecursiveCharacterTextSplitter

8. Evaluation Methodology

The system can be evaluated using two approaches.

1. LLM-as-Judge Evaluation

Generated responses can be evaluated for:

• Correctness
• Groundedness
• Hallucination detection

2. Automated Metrics

Common evaluation metrics include:

• Exact Match (EM) – Exact match with reference answer
• F1 Score – Word-level overlap between predicted and true answers
• Context Recall – Whether the retrieved documents contain the correct answer

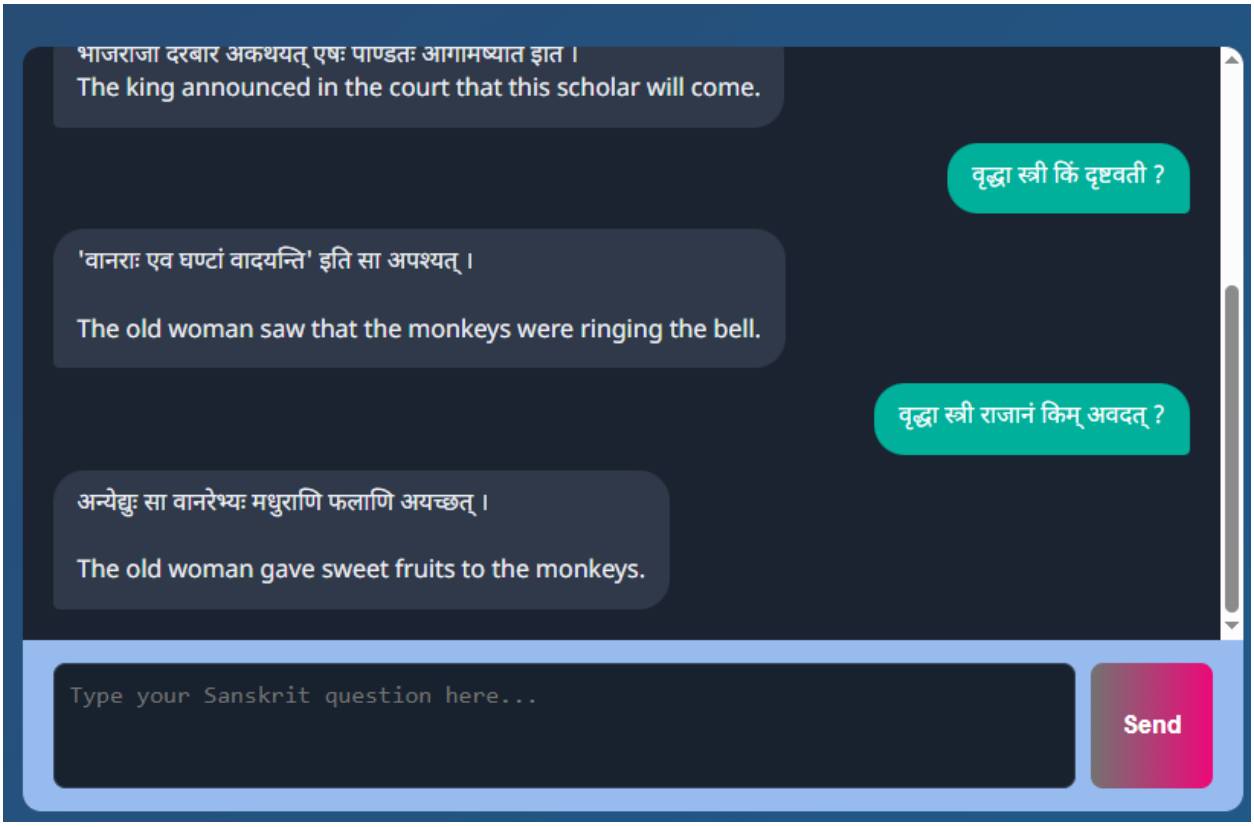
9. Results

The developed system successfully demonstrates the ability to answer Sanskrit questions using document retrieval and language model generation.

Key observations:

• Hybrid retrieval improves answer relevance
• Transliteration support improves usability
• Context grounding reduces hallucinated responses

The web interface allows users to interact with the system in real time and obtain responses generated from Sanskrit documents.



10. Key Features

✓ Hybrid Retrieval (FAISS + BM25)
✓ Sanskrit and Transliteration Query Support
✓ Context-Grounded Answer Generation
✓ Modular RAG Architecture
✓ FastAPI Backend Deployment
✓ Scalable Knowledge Base Design

11. Challenges and Solutions

Challenge	Solution
Handling transliteration	Script detection and conversion module

Challenge	Solution
Hallucinated answers	Retrieval grounding and strict prompting
Poor retrieval accuracy	Hybrid FAISS + BM25 search
Response latency	Efficient retrieval pipeline

12. Conclusion

This project demonstrates the successful development of an **end-to-end Sanskrit Retrieval-Augmented Generation chatbot**. By combining hybrid document retrieval with LLM-based generation, the system produces **context-aware and grounded answers** while minimizing hallucinations.

The modular architecture allows the system to scale to larger document collections and supports future enhancements such as improved evaluation metrics, advanced retrieval methods, and multilingual support.

The approach highlights the potential of RAG systems for **knowledge retrieval and question answering in classical languages such as Sanskrit**.